

SHERPA

Sports, Heritage and Excursion Recommendation and Planning Agent Pilot (Demo) Setup Guide

An ordered, complete setup for the full pilot: local development and cloud demo

Author: Martín Galán Merchán

Repository: github.com/mgalanme/sherpa

Companion package: SHERPA_Setup_Scripts.zip

Version 0.1 (Draft for review) | Date: 13 June 2026

Table of Contents

1. Introduction and How to Use This Document	3
2. The Two-Environment Model	4
2.1 Local Development Environment	4
2.2 Cloud Demo Environment	4
3. Technology Stack Board	5
4. Accounts and Credentials Checklist	7
5. Local Prerequisites	8
6. Deploying the Scripts	9
6.1 Project layout	9
7. Environment Configuration (.env)	10
8. Virtual Environments	11
9. Local Development Stack	12
10. Cloud Services Setup	13
10.1 Databricks	13
10.2 Neo4j Aura Free	13
10.3 Qdrant Cloud Free	13
10.4 Solace Cloud	13
10.5 LangSmith	13
10.6 Streamlit Community Cloud	13
11. Connectivity Checks and Smoke Tests	14
12. GitHub Repository	15
13. Canonical Run Order	16
14. Teardown	17
15. Troubleshooting Appendix	18
Appendix A. Script Inventory	19

1. Introduction and How to Use This Document

This document is the complete, ordered setup guide for the SHERPA pilot. It is written so that everything needed is listed before it is required, so that the build does not stall halfway through for a missing dependency or account. It follows the same approach used for VITRUVIO: the document describes and references the scripts, but does not contain them. The scripts are delivered separately in the companion package SHERPA_Setup_Scripts.zip, together with a deployer that copies them into the project folder.

The pilot is deliberately ambitious and contains every part of the architecture: the conversational interface and Human-in-the-Loop review console, the governed event-driven agent mesh, the classical recommendation models, the data pipelines, and all the open-data integrations, across the full catalogue of activities.

Convention: file names, commands and variables appear in a monospace font, for example `00_deploy.sh` and `SHERPA_HOME`. British English is used throughout, with no em dashes and no separator lines between paragraphs.

Scope: this is the pilot or demo, on free technologies only. Databricks runs on your existing free cloud workspace and Streamlit runs on Streamlit Community Cloud. The production blueprints for AWS, Azure, GCP and Alibaba Cloud are out of scope here and were covered in the Foundational Architecture Document.

2. The Two-Environment Model

Because the deployed application runs on Streamlit Community Cloud, it cannot reach services that run only on your local workstation, which has no public address. The pilot therefore uses two environments that share the same code and the same .env contract, switching behaviour through the DEPLOY_ENV variable.

2.1 Local Development Environment

On your Linux Mint workstation (user pruebas, no sudo, root available via su) you run the full stack for development and for exercising the event mesh faithfully. A Docker stack provides a Solace PubSub+ broker, Qdrant and Neo4j with APOC, and Ollama provides the local sovereign fallback model. The Solace Agent Mesh agents run as local processes against the local broker, which is the most faithful way to develop and debug the mesh.

2.2 Cloud Demo Environment

The shareable demo runs the Streamlit application on Streamlit Community Cloud. The mesh runtime starts in-process within the application and connects to a Solace Cloud broker, so the event mesh genuinely runs in the cloud. The application reaches Neo4j Aura Free for the graph, Qdrant Cloud Free for vectors, and your Databricks workspace for the curated gold tables and the append-only, hash-chained audit trail held in Delta. Groq serves the language model in both environments; Ollama is local only.

Resource note for the free tier: in-process embedding with a transformer model is heavy for Streamlit Community Cloud. The architecture mitigates this by precomputing place-fact embeddings in the Databricks gold pipeline, so that at runtime the application embeds only the short user query. This keeps the deployed app light. A contingency, if the free tier still struggles, is to move the runtime query embedding to a hosted call.

3. Technology Stack Board

This is the board of chosen components, with what each is, why it was chosen, and what it is for. Everything here is free for the pilot.

Component	What it is	Why and what for
Streamlit (Community Cloud)	Python web application framework	Hosts the conversational interface and the HITL review console; free cloud hosting from your existing account
Solace Agent Mesh + PubSub+	Open-source event-driven multi-agent framework and broker	Governed orchestration with agent-to-agent messaging and full observability; local broker for development, Solace Cloud for the demo
LangGraph	Stateful orchestration library	The in-agent Human-in-the-Loop pause and resume, using interrupt-before with a module-level checkpointer
CrewAI	Collaborative multi-agent library	Collaborative sub-tasks where a small crew is clearer than a single agent
Groq API	Hosted LLM inference (free tier)	Primary language model (llama-3.3-70b) for understanding and drafting, with an 8b fallback; reliable function calling
Ollama (qwen2.5:3b)	Local model server	Sovereign local fallback for development; qwen2.5 supports function calling whereas llama3 does not
nomic-embed-text-v1	Embedding model (768 dims)	Grounds retrieval; produced in-process; precomputed in the pipeline for the cloud app
Databricks (free workspace)	Data platform with Delta Lake	Medallion pipelines for open data, the curated gold tables and the hash-chained audit trail
Qdrant Cloud Free	Vector database	Retrieval over place facts; client pinned to the server minor (v1.12.4); use query_points not search
Neo4j Aura Free	Graph database with APOC	Relationships among places, tracks, points of interest and species; set enhanced schema off
LangSmith	Tracing and observability	Traces agent and model calls during development and the demo
GitHub + gh	Source control and CLI	Repository creation and synchronisation via the GitHub CLI
uv	Python environment manager	User-space virtual environments, no root required
ReportLab	PDF generation	Renders the final dossier; pure Python, so it is portable to Streamlit Cloud

3.1 Open Data Sources used by the Pilot

Source	Provides	Key needed
AEMET OpenData	Spain weather: municipal forecasts and warnings	Yes (free registration)
Open-Meteo	Global forecast, ERA5 history, elevation	No
OpenStreetMap Overpass	Access tags, points of interest, restrictions	No (descriptive User-Agent required)
OpenRouteService	Routing and elevation on OSM data	Yes (free key)
Wikipedia and Wikidata	History and structured facts	No (descriptive User-Agent required)
GBIF and iNaturalist	Biodiversity records for flora and fauna	No

4. Accounts and Credentials Checklist

Create or confirm the following accounts before configuring the environment. Where you already hold a key from an earlier project, reuse it; all credentials are stored as environment variables or in a secrets manager, never hardcoded.

Account	Free tier and what to obtain	Where it is used
Databricks	Your existing free workspace; obtain a personal access token, the SQL Warehouse HTTP path and its id	Pipelines, gold tables, audit trail
Streamlit Community Cloud	Your existing account; connect it to the GitHub repository	Hosts the deployed application
Neo4j Aura	Create a Free instance; record the neo4j+s URI, user and password	Graph store for the app
Qdrant Cloud	Create a Free cluster; record the URL and API key	Vector store for the app
Solace Cloud	Create a free messaging service; record the SMF host, VPN, user and password	Cloud event broker for the demo mesh
Groq	Create an API key	Primary LLM in both environments
LangSmith	Create an API key and a project named sherpa	Tracing and observability
GitHub	Confirm gh is authenticated with a PAT	Repository for the project
AEMET OpenData	Register for a free API key	Spanish weather data
OpenRouteService	Register for a free API key	Routing and elevation

No key required: Open-Meteo, OpenStreetMap Overpass, Wikipedia, Wikidata, GBIF and iNaturalist. Wikipedia and OpenStreetMap require a descriptive User-Agent string, set through HTTP_USER_AGENT in the .env file.

5. Local Prerequisites

On the Linux Mint workstation (user pruebas), confirm the following are installed. The script `01_prereqs_check.sh` reports each one and gives a hint where something is missing. Most tooling installs in user space; only Docker may need group membership configured with root via `su`.

Tool	Purpose	Note
Docker	Local containers for Solace, Qdrant and Neo4j	Ensure the daemon is reachable for your user
uv	Virtual environment and dependency manager	Installed user-space by <code>03_uv_bootstrap.sh</code> if missing
Python 3.11 or 3.12	Runtime for all environments	uv can also manage interpreters
gh	GitHub CLI	Run <code>gh auth login</code> with a PAT
git	Version control	Configure <code>git pull --rebase</code> globally
Ollama	Local fallback model server	Pull <code>qwen2.5:3b</code> with <code>08_ollama_models.sh</code>
curl and jq	Cloud connectivity checks	Used by <code>09_cloud_accounts_check.sh</code>
Databricks CLI	Workspace bootstrap	Configured with the host and token from <code>.env</code>

6. Deploying the Scripts

Download and extract `SHERPA_Setup_Scripts.zip` into `/home/pruebas/Descargas`. Then run the deployer, which creates the project folder `/home/pruebas/formacion/sherpa` (it does not exist yet) and copies everything into place.

```
cd /home/pruebas/Descargas/SHERPA_Setup_Scripts
bash scripts/00_deploy.sh
```

The deployer reads an optional `SHERPA_ORIGIN` and `SHERPA_HOME`, defaulting to the extracted folder and to `/home/pruebas/formacion/sherpa` respectively. After it completes, all further work happens from the destination.

6.1 Project layout

Path	Contents
<code>scripts/</code>	The numbered setup scripts (00 to 13) and <code>99_tearardown.sh</code>
<code>scripts/smoke/</code>	The Python connectivity smoke tests
<code>databricks/notebooks/</code>	Placeholder for the Medallion notebooks (delivered with the application code)
<code>data/gpx/</code> , <code>data/outputs/</code>	Local GPX inputs and generated dossiers
<code>src/</code>	Placeholder for the application source (delivered separately)
<code>.env</code> , <code>.env.example</code>	The completed configuration and its template
<code>requirements-*.txt</code>	Per-environment dependency lists, plus the merged cloud list
<code>docker-compose.yml</code>	The local development container definitions

7. Environment Configuration (.env)

Run `02_dirs_and_env.sh` to create `.env` from the template. It never overwrites an existing file. Open `.env` and complete every value. The principal variables are described below.

Variable	Meaning
DEPLOY_ENV	local for development, cloud for the deployed demo
GROQ_API_KEY, GROQ_MODEL_PRIMARY	Groq credentials and the primary model (llama-3.3-70b-versatile)
OLLAMA_HOST, OLLAMA_MODEL	Local model server and model (qwen2.5:3b); development only
EMBED_MODEL, EMBED_DIM	Embedding model (nomic-ai/nomic-embed-text-v1) and dimension (768)
NEO4J_URI, NEO4J_PASSWORD	Neo4j Aura connection for the cloud app
NEO4J_LOCAL_URI, NEO4J_LOCAL_PASSWORD	Local Neo4j for development
QDRANT_URL, QDRANT_API_KEY	Qdrant Cloud connection; local URL for development
DATABRICKS_HOST, DATABRICKS_TOKEN, DATABRICKS_HTTP_PATH, DATABRICKS_WAREHOUSE_ID	Databricks workspace, token and SQL Warehouse
DATABRICKS_CATALOG, DATABRICKS_SCHEMA, DATABRICKS_SECRET_SCOPE	Catalog, schema and secret scope names
SOLACE_HOST, SOLACE_VPN, SOLACE_USERNAME, SOLACE_PASSWORD	Solace Cloud broker for the deployed mesh
SOLACE_LOCAL_*	Local PubSub+ broker for development
LANGCHAIN_API_KEY, LANGCHAIN_PROJECT	LangSmith tracing
AEMET_API_KEY, ORS_API_KEY, OVERPASS_URL, HTTP_USER_AGENT	Open-data access
GH_REPO, GH_BRANCH	GitHub repository and branch

8. Virtual Environments

Three isolated environments are created with `uv`, so that the dependency footprints do not conflict. This is a deliberate refinement over a two-environment setup, because Solace Agent Mesh pulls in the Solace AI Connector and the Google Agent Development Kit, which are best kept apart from the LangChain and CrewAI stacks.

Environment	Created by	Holds
<code>.venv-langchain</code>	<code>04_venv_langchain.sh</code>	Orchestration, data access, classical AI, embeddings and the Streamlit application
<code>.venv-crewai</code>	<code>05_venv_crewai.sh</code>	The CrewAI collaborative crews
<code>.venv-mesh</code>	<code>06_venv_mesh.sh</code>	The Solace Agent Mesh runtime and the Solace messaging API

Always run from the project root with `PYTHONPATH=. set`, and run `ruff check` and `ruff format` before every commit. When using CrewAI, import its modules in the main thread before spawning worker threads.

For Streamlit Community Cloud, which supports a single environment, the merged and pinned `requirements-cloud.txt` is used. Copy it to `requirements.txt` at the repository root before deploying.

9. Local Development Stack

Start the local containers with `07_docker_compose_up.sh`. The image tags are pinned, never latest, and the Qdrant tag matches the pinned client. Allow up to a minute for the Solace broker to become ready.

Service	Image (pinned)	Local endpoint
Solace PubSub+	<code>solace/solace-pubsub-standard:10.10</code>	Manager <code>http://localhost:8080</code> ; SMF 55555
Qdrant	<code>qdrant/qdrant:v1.12.4</code>	<code>http://localhost:6333/dashboard</code>
Neo4j with APOC	<code>neo4j:5.24-community</code>	<code>http://localhost:7474</code> ; Bolt 7687

Then pull the local model with `08_ollama_models.sh`, which fetches `qwen2.5:3b`. Embeddings are produced in-process with `nomic-embed-text-v1`, so no embedding model is pulled.

10. Cloud Services Setup

Provision the cloud services in the following order. Each step records values into `.env`, and the connectivity is verified later by the smoke tests.

10.1 Databricks

In your free workspace, confirm a SQL Warehouse exists and record its HTTP path and id. Configure the Databricks CLI with the host and token. Then run `10_databricks_bootstrap.sh`, which creates the secret scope, stores the reused keys, and creates the catalog, schema and the append-only, hash-chained `audit_event` Delta table. The Medallion ingestion notebooks and any scheduled job are delivered with the application code; this step prepares the workspace to receive them.

10.2 Neo4j Aura Free

Create a Free instance, accept the generated password, and record the neo4j+s URI, the user and the password into `NEO4J_URI`, `NEO4J_USERNAME` and `NEO4J_PASSWORD`. Aura includes APOC core, which is sufficient for the pilot. Set the enhanced schema flag off in the application code, in line with the established lesson.

10.3 Qdrant Cloud Free

Create a Free cluster and record the URL and API key into `QDRANT_URL` and `QDRANT_API_KEY`. Confirm the cluster version is compatible with the pinned client v1.12.4. The application uses `query_points` rather than the deprecated `search`.

10.4 Solace Cloud

Create a free messaging service. From the connection details, record the secured SMF host, the message VPN, the client username and password into `SOLACE_HOST`, `SOLACE_VPN`, `SOLACE_USERNAME` and `SOLACE_PASSWORD`. This broker carries the agent-to-agent messages of the deployed mesh.

10.5 LangSmith

Create an API key and a project named `sherpa`, and record `LANGCHAIN_API_KEY` and `LANGCHAIN_PROJECT`. Tracing is enabled through `LANGCHAIN_TRACING_V2`.

10.6 Streamlit Community Cloud

Connect the GitHub repository to your Streamlit account and create an app pointing at the application entry point (delivered with the application code). Set the application secrets to mirror the cloud values in `.env`. Ensure the repository root contains a `requirements.txt` equal to `requirements-cloud.txt`. No `packages.txt` is required because PDF generation uses ReportLab. Remember the resource note in section 2: place-fact embeddings are precomputed in Databricks so the deployed app stays light.

11. Connectivity Checks and Smoke Tests

Validate credentials and reachability before running the application. First run `09_cloud_accounts_check.sh`, which confirms the cloud variables and performs light HTTP checks against Groq, Qdrant Cloud, Databricks, LangSmith and AEMET. Then run `11_solace_cloud_check.sh` to confirm the Solace Cloud target. Finally run `12_smoke_test.sh`, which executes the driver-level tests in the correct environments.

Smoke test	Environment	Verifies
<code>smoke_graph.py</code>	<code>.venv-langchain</code>	Neo4j connectivity and APOC
<code>smoke_vector.py</code>	<code>.venv-langchain</code>	Qdrant connectivity and collections
<code>smoke_llm.py</code>	<code>.venv-langchain</code>	Groq response and the local Ollama model
<code>smoke_databricks.py</code>	<code>.venv-langchain</code>	Databricks SQL Warehouse query
<code>smoke_mesh.py</code>	<code>.venv-mesh</code>	Solace broker connect and publish

A single verdict is printed at the end. Any failure points to the Troubleshooting appendix.

12. GitHub Repository

Create or update the repository with `13_github_sync.sh`, which generates a correct `.gitignore` if it is missing, runs `ruff` only on source directories that exist, commits with an optional message, performs a rebasing pull and pushes. It uses `gh` and your authenticated session.

```
bash scripts/13_github_sync.sh "feat: pilot setup complete"
```

13. Canonical Run Order

This is the single ordered checklist for a clean build, so that nothing is forgotten.

1. Extract the ZIP into `/home/pruebas/Descargas` and run `scripts/00_deploy.sh`.
2. `cd /home/pruebas/formacion/sherpa/scripts`.
3. `01_prereqs_check.sh`, then resolve anything reported.
4. `02_dirs_and_env.sh`, then complete every value in `.env`.
5. `03_uv_bootstrap.sh`, then open a new shell or source `~/ .bashrc`.
6. `04_venv_langchain.sh`, `05_venv_crewai.sh`, `06_venv_mesh.sh`.
7. For local development only: `07_docker_compose_up.sh`, then `08_ollama_models.sh`.
8. `09_cloud_accounts_check.sh`.
9. `10_databricks_bootstrap.sh`.
10. `11_solace_cloud_check.sh`.
11. `12_smoke_test.sh`, and confirm the verdict is OK.
12. `13_github_sync.sh` to publish the repository.
13. Configure the Streamlit Community Cloud app and its secrets (section 10.6).
14. At the end of a session, `99_teardown.sh` to release local resources.

14. Teardown

Run `99_teardown.sh` to stop the local containers (volumes are preserved) and to stop any Streamlit, mesh or uvicorn processes. To also delete the local container data, use the `down` command with the `volumes` flag shown by the script.

15. Troubleshooting Appendix

These are the recurring issues and the lessons that prevent them, gathered from earlier case studies.

- Qdrant version mismatch: pin the client to the same minor as the server (v1.12.4) and use `query_points()` rather than the deprecated `search()`.
- Neo4j graph operations: set `enhanced_schema=False` in the `Neo4jGraph` constructor, and ensure APOC is available (it is, on Aura core and in the Docker image).
- LangGraph naming: node names must not collide with state-key names, or the workflow will misbehave; rename the node, for example to `run_contrast`.
- LangGraph HITL: use `interrupt_before` with a module-level checkpoint singleton, not one created per invocation, so the workflow can resume after review.
- CrewAI threads: import CrewAI modules in the main thread before spawning worker threads.
- Groq free-tier limits: mitigate with heuristic pre-filtering instead of an LLM call, serialised workers, and short sleeps between calls.
- Ollama model: use `qwen2.5:3b`, which supports function calling; `llama3` does not.
- Always set `PYTHONPATH=.` from the project root, and run `ruff check` and `ruff format` before each commit, not after a failed one.
- Configure `git pull --rebase` globally to avoid divergent branches across machines.
- Streamlit Cloud cannot reach local services; the deployed app must use the cloud endpoints (Aura, Qdrant Cloud, Databricks, Solace Cloud).
- Embedding weight on the free tier: precompute place-fact embeddings in Databricks and embed only the short user query at runtime.
- Solace broker startup: allow up to a minute for the local PubSub+ container to become ready before connecting.
- Files containing triple backticks cannot be created through a shell here-document; deliver them as downloadable files.

Appendix A. Script Inventory

Script	Purpose
00_deploy.sh	Deploy scripts from Descargas to formacion/herpa
01_prereqs_check.sh	Verify local prerequisites
02_dirs_and_env.sh	Create layout and the .env from template
03_uv_bootstrap.sh	Ensure uv in user space
04_venv_langchain.sh	Create the .venv-langchain environment
05_venv_crewai.sh	Create the .venv-crewai environment
06_venv_mesh.sh	Create the .venv-mesh environment
07_docker_compose_up.sh	Start local containers (Solace, Qdrant, Neo4j)
08_ollama_models.sh	Pull qwen2.5:3b for local development
09_cloud_accounts_check.sh	Check cloud credentials and connectivity
10_databricks_bootstrap.sh	Secret scope, catalog, schema and audit table
11_solace_cloud_check.sh	Validate the Solace Cloud target
12_smoke_test.sh	Run the end-to-end smoke tests
13_github_sync.sh	Create or update the GitHub repository
99_teardown.sh	Stop local containers and processes

End of document. *On approval, the next deliverable will be the application code package, containing the Medallion notebooks, the agent definitions, the classical models and the Streamlit application.*